

State Machines in VHDL

You should already have seen the basics of VHDL in a previous course (ECE 3380).

Today we will simply provide the basic syntax for implementing a control logic state machine in VHDL.

State Machines (cont).

As you will remember, all VHDL logic begins with the ENTITY declaration.

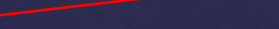
```
ENTITY control_vhdl is
PORT (
  START : IN STD_LOGIC;
  YZero : IN STD_LOGIC;
  Y0     : IN STD_LOGIC;
  Y1     : IN STD_LOGIC;
  RESET  : IN STD_LOGIC;
  CLK    : IN STD_LOGIC;

  LDX    : OUT STD_LOGIC;
  SHX    : OUT STD_LOGIC;
  LDY    : OUT STD_LOGIC;
  SHY    : OUT STD_LOGIC;
  LDZ    : OUT STD_LOGIC;
  CLRZ   : OUT STD_LOGIC;
);
END control_vhdl;
```

You always need RESET and CLK



You will usually have some inputs and outputs



Next is the ARCHITECTURE declaration.

Note BEGIN, END Entity name goes here

Create a State_type. All states must be named in parentheses

```
ARCHITECTURE RTL of control_vhdl IS
TYPE State_type IS (INIT,PARSE,DO,DONT);
SIGNAL State : State_type;
SIGNAL Outputs : STD_LOGIC_VECTOR (5 downto 0);
BEGIN
-- The next state logic is in the PROCESS block
PROCESS (CLK,RESET)
-- Since this is a Moore machine, the output logic is a function of State only
-- In the case of a Mealy machine, the output PROCESS is also a function of input
PROCESS (State)
(LDX,SHX,LDY,SHY,LDZ,CLRZ) <= Outputs;
END RTL;
```

I have collapsed the guts of the logic to focus on the key elements of the ARCHITECTURE

I recommend you create two concurrent PROCESS blocks

This block handles “next state” logic

```
ARCHITECTURE RTL of control_vhdl IS
TYPE State_type IS (INIT,PARSE,DO,DONT);
  SIGNAL State : State_type;
  SIGNAL Outputs : STD_LOGIC_VECTOR (5 downto 0);

BEGIN
  -- The next state logic is in the PROCESS block
  PROCESS (CLK,RESET)
  -- Since this is a Moore machine, the output logic is a function of State only
  -- In the case of a Mealy machine, the output PROCESS is also a function of input
  PROCESS (State)
    (LDX,SHX,LDY,SHY,LDZ,CLRZ) <= Outputs;
  END RTL;
```

This block handles the output logic

Next state logic

```
PROCESS (CLK,RESET)
BEGIN
  IF (RESET = '1') THEN
    State <= INIT;

  ELSIF rising_edge(CLK) THEN
    CASE State IS
      WHEN INIT =>
        IF START='1' THEN
          WHEN PARSE =>
            IF Y0='1' THEN
              State <= D0;
            ELSE
              State <= DONT;
            END IF;

            WHEN D0 =>
              IF YZero = '1' THEN
                ELSIF Y1='0' THEN

              WHEN DONT =>
                IF YZero='1' THEN
                ELSIF Y1='1' THEN

            END CASE;
          END IF;
        END PROCESS;
```

Next state logic triggers on changes to CLK,RESET only

Initialization on RESET = '1'

State change on rising clock only

Case statement is a good way to make sure you handle all states

Transition logic

Note syntax

Next state logic

```
-- Since this is a Moore machine, the output logic is a function of State only
-- In the case of a Mealy machine, the output PROCESS is also a function of input
PROCESS (State)
BEGIN
  Case State IS
    WHEN INIT => Outputs <= "111100";
    WHEN PARSE => Outputs <= "000001";
    WHEN DO => Outputs <= "010110";
    WHEN DONT => Outputs <= "010100";
  END CASE;
END PROCESS;

(LDX,SHX,LDY,SHY,LDZ,CLRZ) <= Outputs;
```

Syntax

More syntax

Once again, Case
makes it easy to
remember all states

This PROCESS only changes on
changes to State: Moore machine

For the course project (to be introduced later) you will have to implement control logic in VHDL for a simple processor

I will provide the datapath and the instruction set.